

Tendências de Motores de Jogos

Semana 5

Blueprint Interfaces e Event Dispatchers

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 2 — Construir Sistemas

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Compreender a comunicação desacoplada entre sistemas como problema universal de arquitetura
- Diferenciar Blueprint Interface (contrato) de Event Dispatcher (notificação de evento)
- Implementar `BPI_Interactable` e conectá-la ao `InteractionComponent` de `BP_Player`
- Implementar um Event Dispatcher acionado por interação, aplicado a um objeto interativo concreto

Como a semana está organizada



Encontro 1

Blueprint Interface `BPI_Interactable` como contrato de comunicação



Encontro 2

Event Dispatcher, objeto interativo concreto e primeiro Feedback Formal da disciplina

Tendências de Motores de Jogos

ENCONTRO 1

Blueprint Interface: BPI_Interactive

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Como o InteractionComponent do jogador pode "chamar" uma porta, uma alavanca e um baú sem conhecer nenhum dos três?

Pense no que aconteceria se cada objeto exigisse um bloco de decisão próprio no Player.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Desacoplamento por contrato

- Sistema chamador (InteractionComponent) precisa agir sobre sistemas diferentes sem conhecer a implementação de cada um
- A engine garante apenas que o alvo "responde a uma chamada esperada" — nunca como
- Sem esse contrato, o InteractionComponent precisaria de um bloco de decisão por tipo de objeto, crescendo a cada novo interativo



DICA

Problema universal: todo sistema de comunicação entre objetos independentes precisa de um contrato, não de conhecimento mútuo.

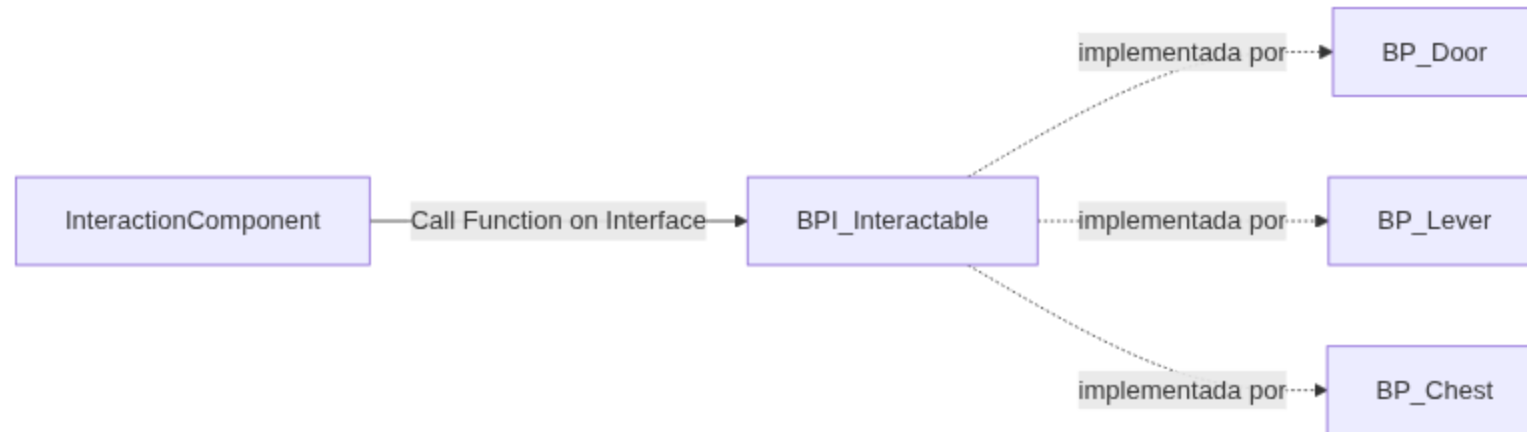
Blueprint Interface: o contrato da Unreal

- Interface declara funções sem implementação — nenhuma lógica dentro dela
- Qualquer Actor pode implementá-la, sendo chamado de forma genérica por quem só conhece o contrato
- `Call Function on Interface` chama a função sem exigir conversão para a classe concreta
- Interface nunca é instanciável por si só

DICA

A interface nunca carrega estado nem lógica — apenas declara o que deve existir. A lógica pertence inteiramente a quem implementa.

Contrato × implementação concreta



Contrato de comunicação: Unreal × Unity

Unreal

Blueprint Interfaces funcionam nativamente em visual scripting; `Call Function on Interface` chama sem referência de classe concreta

Unity

Interfaces em C# resolvem o mesmo problema; tipicamente exige `GetComponent<IInteragivel>()` para obter a referência tipada, em código

Demonstração: criando BPI_Interactable

O professor cria a Blueprint Interface `BPI_Interactable` em `Interactables/`, com uma função `Interact` sem corpo de execução, e a implementa em um Actor de demonstração.

Resultado esperado: o InteractionComponent chama `Interact` via "Call Function on Interface" sobre o Actor de demonstração, sem conhecer sua classe concreta.

Imagem sugerida

*Objetivo: mostrar o painel Class Settings de um Actor com **BPI_Interactable** adicionada em Implemented Interfaces.*

Enquadramento: editor de Blueprint do Actor de demonstração, painel Class Settings visível com a lista de interfaces implementadas.

*Elementos presentes: **BPI_Interactable** destacada na lista; aba Functions ao lado mostrando **Interact** disponível sob a seção Interfaces.*

Destaque visual: contorno ao redor da interface na lista de Implemented Interfaces.

Legenda sugerida: "Implementar uma interface não copia comportamento — apenas garante que a função exigida pode ser chamada."

Arquitetura: onde a interface mora

- `Blueprints/Interactables/BPI_Interactable` — primeira classe da subpasta `Interactables/`
- Convenção de nomenclatura `BPI_` conforme `PROJECT_ARCHITECTURE.md`, seção 9
- Base para `BP_Door`, `BP_Lever`, `BP_Chest`, `BP_Pickup` e `BP_Checkpoint` nas próximas semanas

NA INDÚSTRIA

Toda interação futura do Vertical Slice — porta, baú, alavanca, checkpoint — reutiliza este mesmo contrato, sem refatoração.

Boas práticas

✓ BOAS PRÁTICAS

- Manter a interface com o menor número possível de funções — apenas o que precisa ser chamado de forma genérica
- Nunca duplicar lógica dentro da interface; a lógica pertence ao Actor que implementa
- Preferir "Call Function on Interface" a qualquer chamada direta pela referência de classe concreta

Laboratório do Encontro 1

Criar `BPI_Interactive` em `Interactables/` e implementá-la em ao menos um Actor do nível de teste, com o `InteractionComponent` chamando `Interact` via interface.

OBJETIVOS

Critério de sucesso: Actor implementando `BPI_Interactive` reagindo de forma simples e visível (Print String ou cor) quando chamado pelo `InteractionComponent`, com nomenclatura conforme `PROJECT_ARCHITECTURE.md`.

Fechando o Encontro 1

- Blueprint Interface é contrato de comunicação sem conhecimento de classe concreta
- `BPI_Interactable` criada e implementada em ao menos um Actor do nível de teste
- Próximo encontro: como o objeto ativado avisa sua própria reação, sem conhecer quem escuta

Tendências de Motores de Jogos

ENCONTRO 2

Event Dispatcher e o objeto interativo

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos



**Se uma porta precisar avisar sua animação,
um sistema de áudio e uma missão ao
mesmo tempo, ela deveria conhecer os três?**

Pense em como um objeto notifica vários sistemas sem manter uma lista fixa de quem reagir.

Event Dispatcher: o padrão observer

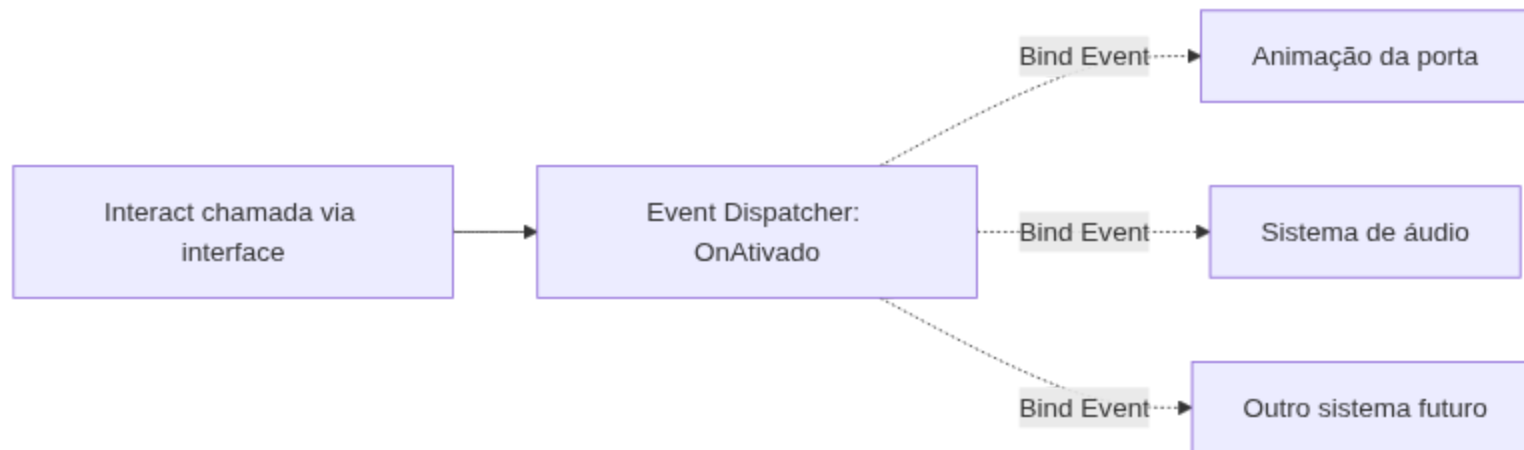
- Resolve o problema inverso da interface: quem precisa ser avisado quando algo acontece, sem que o emissor conheça os interessados
- Um objeto declara um evento (ex.: "fui ativado"); qualquer sistema pode se inscrever para reagir
- **Bind Event to** inscreve uma reação; o disparo ocorre dentro da função de interface já garantida pelo contrato



DICA

Interface resolve "quem pode ser chamado sem conhecer a classe"; Event Dispatcher resolve "quem precisa ser avisado sem que o emissor conheça os inscritos".

Emissor e inscritos



Notificação de eventos: Unreal × Unity

Unreal

Event Dispatchers configuráveis visualmente no Blueprint Editor, conectados via `Bind Event to` sem escrever código

Unity

UnityEvent (via Inspector ou código) e C# Actions (`Action` , `Action<T>`); inscrição via `AddListener` ou `+=`

Demonstração: Event Dispatcher acionado pela interface

O professor cria um Event Dispatcher no Actor de demonstração, dispara-o dentro da função `Interact` e conecta (`Bind Event to`) uma reação simples, sem que o Actor conheça quem reage.

Resultado esperado: o Actor dispara a notificação; a reação ocorre por inscrição, nunca por chamada direta.

Imagem sugerida

Objetivo: mostrar o Event Graph do Actor de demonstração com o Event Dispatcher disparado dentro de Interact e a reação conectada via Bind Event.

Enquadramento: Event Graph completo do Actor, com os dois blocos (disparo e inscrição) visíveis lado a lado.

Elementos presentes: nó "Call OnAtivado" dentro da função Interact; nó "Bind Event to OnAtivado" conectado a uma reação simples (mudança de cor ou movimento).

Destaque visual: seta ligando o disparo à inscrição, reforçando que são dois pontos distintos do grafo.

Legenda sugerida: "O Actor dispara a notificação; a reação pertence a quem se inscreve, não a quem dispara."

Arquitetura: Interactables consolidado

- `BPI_Interactive` e os Event Dispatchers de interação tornam-se a base de comunicação reutilizada por todo objeto interativo futuro
- `BP_Door`, `BP_Lever`, `BP_Chest`, `BP_Pickup` e `BP_Checkpoint` (próximas semanas) reutilizam a mesma estrutura, sem refatoração
- Mecanismo de acionamento é decisão de design do grupo; a estrutura de comunicação é fixa

NA INDÚSTRIA

A mesma dupla Interface + Event Dispatcher construída hoje será reaproveitada para baús, checkpoints e pickups, sem retrabalho.

Boas práticas

✓ BOAS PRÁTICAS

- Nomear o Event Dispatcher de forma descritiva (`OnAtivado` , `OnPortaAberta`), nunca genérica (`Event1`)
- Disparar o Dispatcher sempre dentro da função que a interface já garante ser chamada de forma desacoplada
- Remover nós de teste temporários (Print String) antes da apresentação do Feedback Formal

Desafio: objeto interativo com mecanismo próprio

Cada grupo implementa um objeto interativo (porta ou equivalente) combinando `BPI_Interactable` + Event Dispatcher, com liberdade total sobre o mecanismo de acionamento — alavanca, chave, proximidade ou solução própria.

ATENÇÃO

O objeto deve reagir de forma demonstrável a uma ação do jogador via `InteractionComponent`. Sem gabarito único para o mecanismo de acionamento.

Feedback Formal

Cada grupo apresenta seu mecanismo de acionamento e recebe o primeiro Feedback Formal registrado da disciplina.

OBJETIVOS

Critério de avaliação: funcionamento demonstrável do objeto interativo, clareza da justificativa do mecanismo escolhido e organização/nomenclatura conforme PROJECT_ARCHITECTURE.md.

Resumo da Semana 5

- Blueprint Interface é contrato; Event Dispatcher é notificação — peças complementares, não substituíveis
- Objeto interativo funcional integrado ao Vertical Slice, com mecanismo de acionamento definido pelo grupo
- Primeiro Feedback Formal da disciplina concluído
- Base de comunicação pronta para todo interativo futuro (baús, checkpoints, pickups)

Próximos passos

DICA

A Semana 6 reutiliza o mesmo par Interface + Event Dispatcher para modelar itens coletáveis (baús, moedas, recursos) via Data Assets, Data Tables, Structs e Enums, separando dados de design da lógica de gameplay.

Leitura recomendada: Blueprints Visual Scripting in Unreal Engine (Epic Games Documentation).